

Table 1: Best practices in Jenkins

Best practice	Description
Role based access	This is related to authentication and authorisation of users, and decides who has access and to what extent
Master-agent architecture	In this architecture, the master node does not carry out any build on itself
Backup	Take regular backups of Jenkins Home. Apart from the builds and build logs, there are many other things that are too important to lose, in case anything happens to the instance or system
Naming conventions	Don't use lengthy project names. Jenkins makes a folder for each project/job in the workspace and jobs directory of Jenkins Home; to name them it uses the project's name and even branch name in case a repository is in the picture
File fingerprinting	Make use of file fingerprinting to manage job dependency. Jenkins stores file fingerprints to keep a track on which version of a file is used in a particular job, so as to manage the job dependencies
Builds	Clean builds are the most reliable, when built directly from Source Code Control. This makes sure there is no unwanted part tagged along with the new build. Also, it enforces another good practice that all external jars, release notes, scripts, and so on are checked into the source code repository
Use of collaboration tools	To reduce the maintenance of Change Log, integrate tightly with the issue tracking tools such as JIRA
Testing	Always configure the job to produce trend results and perform automated testing when running a build
Disk space	Ensure Jenkins Home is located on a partition that has enough free space. Each job can be built several times, and each time a separate build data is stored. Apart from this, the artifacts, logs, settings, configuration details, and so on are all stored in Jenkins Home
Archiving jobs	Archive the unused jobs rather than removing them. Archiving makes it possible to resurrect the jobs when in need
Build jobs for branches	Create different jobs for different branches of the same project's repositories or to manage automation with few jobs, use parametrised jobs based on branches
Resource management	In case of multiple jobs running simultaneously, forbid resource collisions. Often jobs can use the same resource such as a port, service, database, and so on. This can lead to build failure. So, to avoid this, allocate different resources if possible, such as ports, or avoid parallel building
Job scheduling	Do not schedule all the jobs to start at the same time
Notifications	Use email/Slack notifications mapped to respective stakeholders such as developers/testers/managers
Tags	Label, tag or baseline the code base after the build success in code repository
Stashing /unstashing files – Blue Ocean	Prefer the stashing of files rather than archiving them: Stashing /unstashing files enables the file sharing between different stages and nodes of the pipeline whereas in archiving, it restricts the file sharing within the same node
Type of pipeline	Use a declarative pipeline rather than a scripted pipeline

and advanced the application life cycle of software organisations by means of automation. It comes with a lot of features and even has the provision to add more, with the help of plugins. Listed below are some of the features Jenkins has to offer:

- Easy to install
- Open source
- Easy to configure
- Supports customisation and extensibility

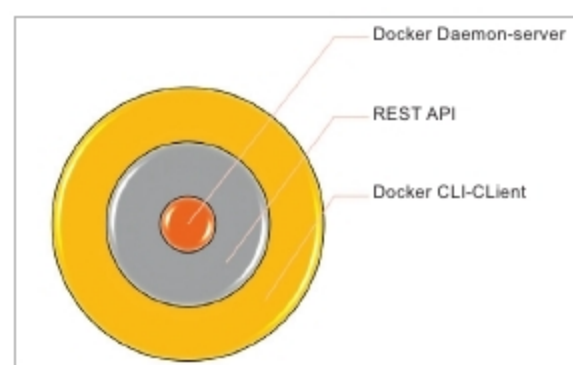


Figure 2: Components of a Docker engine

- Abundance of plugins with well-defined documentation

- Creates custom plugins
- Supports build pipelines
- Facilitates Pipeline as a Code (Blue Ocean)
- Supports Slack/email notifications
- Cron scheduling
- Provides master-agent architecture for distributed builds
- Faster updates
- User interface is simple and clear
- Role based access
- Supports promoted builds